

41052

Advanced Algorithms

Assignment 1 - Zihuang Huang 24970865

A* Search Algorithm | Track B

Spring 2026

Algorithm Overview

For this assignment, I have chosen the A* search algorithm. The A* algorithm is an algorithm mainly used for pathfinding the shortest route between a starting point and the ending point, or the goal. The A* algorithm is itself an expansion upon the widely known Dijkstra's algorithm, with the addition of a heuristic which estimates the remaining 'cost' of the route to the ending point. This addition of a heuristic allows the A* algorithm to prioritise directions moving towards the ending point instead of exploring in every direction, even ones that head away from the ending point. Where Dijkstra's algorithm thinks how far or how costly the current route is, the A* algorithm also takes into consideration how far away the goal may be from where the current route is.

Because the A* algorithm is commonly used for navigation, pathfinding and shortest path problems, I implemented it as an interactive CLI maze puzzle solver. The tool utilises the A* algorithm, in conjunction with several different heuristic functions, to find and visualise the shortest path through user provided mazes whilst comparing and contrasting the performances of the different methods. The tool supports 3 different heuristic functions, those being manhattan distance, euclidean distance and a zero heuristic, which effectively provides Dijkstra's algorithm as a baseline due to the fact that it does not take into account the estimated remaining distance and only prioritises through current path cost.

Key Design Decisions

As the A* maze solver was designed to be a small portable tool and because I was most comfortable with the language, I chose Python for the coding language. As for the user interface, a clean selection screen with clearly numbered selections was prioritised so the user would be able to perform whatever actions they desired quickly and efficiently. The mazes were to be displayed exactly as it appears in their respective text files, with changes applied for which cells were explored by the algorithm and which cells make up the final path from start to finish, allowing the user to clearly see the approach of the different heuristic methods.

To run the tool, you will need to have python installed, navigate to the folder where the tool is located in the terminal, and run the command "Python -m maze_solver". This will bring up the menu where you can access all features of the tool. External mazes can be created and placed into the tool's 'mazes' folder, where they will be ready for immediate use. If there are any errors with custom mazes, the tool will indicate what the errors are with the particular maze.

Apart from the base maze solver, there were also extra features added, including a list to view all available mazes and their specifications and settings to turn on and off specific debug information displays, such as maze dimensions, showing explored cells and statistics for path length. Another major addition to the tool would be the maze creator, which allows users to create their own mazes in the terminal and to save it to the mazes folder for use immediately. Users are also able to make their own mazes, either via notepad or other external editors, as long as they use the correct denotations for the different elements of the maze and fulfil the basic requirements, such as having only 1 starting point and 1 ending

point, as well as a reachable path between the starting and ending points. All you have to do is to place the created maze into the mazes folder and it will be ready to go.

Many of the design decisions included considerations for user experience. The tool initially had no depictions for explored cells and final path, which had to be manually added. In the beginning, explored cells would be denoted by an 'x', whereas the chosen path would be denoted by an 'o' as I assumed these letters would stand out the most when looking at the terminal display. However, through testing, I quickly figured out that the 'o' and 'x' looked way too similar next to each other for larger mazes, and I decided to settle with a comma for explored cells, as it looks very similar to the period used for empty cells, and I kept the 'o' for the final path. Another point of consideration were the navigation for the menus themselves, and many menus would simply display the settings one at a time or not have the confirmation or selections required. The option to return to the main menu was added to most selections to allow the user to quickly back out if they no longer wanted to perform an operation or if they entered the incorrect command. Input validation also had to be added to these selection prompts, for example, limitations on the dimensions of creatable mazes, selectable options as well as default choices if nothing was entered by the user.

Track Specific Writeup

The chosen track of this assignment is track B, which requires the development of a working tool with a user interface. The tool developed for this track is the A* algorithm maze solver. The maze solver is an interactive CLI tool which solves provided mazes, finding the shortest distance between the starting and ending points within the maze through the use of a pathfinding algorithm, which in this case is the A* algorithm. The tool also allows users to see the differences between different heuristic functions when used with the A* algorithm, to compare their efficacy and performance with different maze dimensions and routes.

The maze solver represents each walkable cell as a node in a graph, with edges connecting cells both horizontally and vertically, simulating the 4 directions of movement you can possibly have. The A* algorithm operates on this graph to find a path from the starting point to the ending point. During the search, the A* algorithm maintains a frontier of possible cells to expand from, and assigns each of these cells a priority rating using the function $f(n) = g(n) + h(n)$, where $f(n)$ is the total estimated cost from the current cell to the ending cell, $g(n)$ is the actual cost from the starting cell to the current cell, and $h(n)$ represents the estimated cost from the current cell to the ending cell. This estimate changes when different heuristic functions are applied. The default heuristic used within the tool is Manhattan distance, calculated through the function $h(n) = |\text{row}_n - \text{row}_{\text{goal}}| + |\text{column}_n - \text{column}_{\text{goal}}|$. Other heuristic functions were not implemented, such as the Chebyshev distance, as diagonal movement is not allowed, and all movement has the same weighting as you are simply moving from cell to cell, with no consideration for weights or 'terrain' in a more realistic scenario. This could potentially be a good route for future development on this project, adding diagonal travel to the mazes or to add different terrain with different weightings.

The A* algorithm prioritises the cell with the lowest total estimated cost for the route, represented by $f(n)$. This balances the distance that has already been travelled with the estimated distance to the ending point. As a result, the A* algorithm generally focuses on

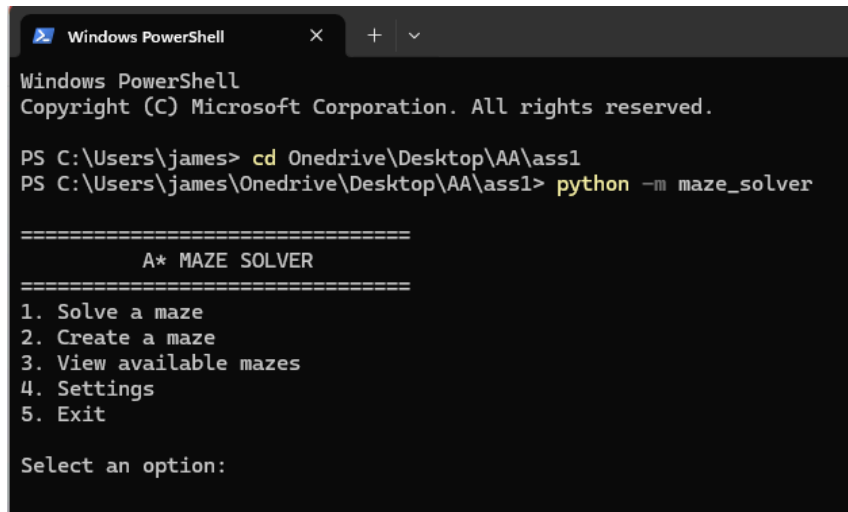
searching towards the goal whilst still taking into account alternative routes when obstacles such as the maze walls are in the way.

Some of the interface decisions have already been discussed within the key design decisions section, and includes the denotation choices for the maze. Periods were used for empty spaces within the maze files themselves as they take up the least amount of space whilst avoiding errors that may arise from using whitespace, such as trailing spaces being removed automatically by certain external editors, difficulty between distinguishing intentional empty cells from accidental whitespaces and visualisations when creating mazes and debugging. However, to improve the readability of maze displays within the terminal, through `Maze.render()`, the periods are swapped with white spaces when displayed to the user. For explored cells where the algorithm have gone, as it is comparatively less important than the final route, I have chosen a comma to denote it. It is easily distinguishable from the period whilst also being small enough to not be confused with the chosen path.

There also had to be a lot of considerations put into the user interface and user experience, with options to go back to the main menu implemented in almost every user prompt in case the user chose the wrong action or changed their mind. Another example of this would be the placement for maze information, statistics and debug information to be at the bottom of the results, as some mazes are so big you are not able to scroll up and see these statistics were they to be placed above the rendered mazes. This is especially an issue when comparing all heuristic functions side by side, as the terminal needs to render 3 of the chosen mazes.

Maze Solver Walkthrough & Screenshots

Firstly, the python command 'python -m maze_solver' is executed within the same folder as where the project is in order to start the tool. You will be presented with the main menu for the maze solver.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

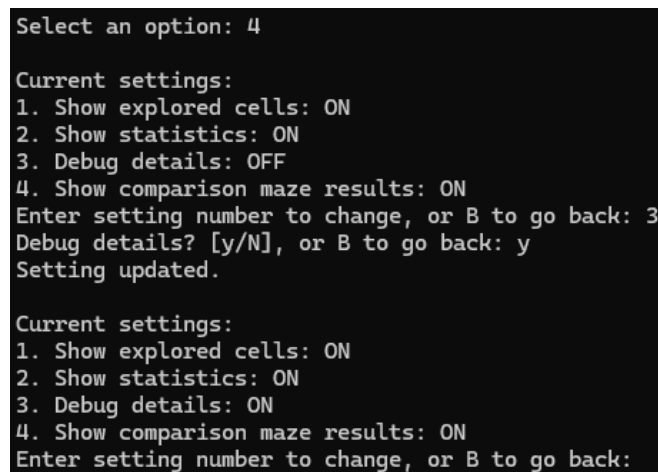
PS C:\Users\james> cd Onedrive\Desktop\AA\ass1
PS C:\Users\james\Onedrive\Desktop\AA\ass1> python -m maze_solver

=====
                A* MAZE SOLVER
=====
1. Solve a maze
2. Create a maze
3. View available mazes
4. Settings
5. Exit

Select an option:
```

Figure WT1: Maze Solver Startup within Powershell and Main Menu

From the main menu, you are able to change settings for the maze solver tool. For example, here I have chosen to turn on debug information which allows us to see information about the maze itself. After changing a setting, it will stay within the settings menu in case you want to change multiple settings at a time.



```
Select an option: 4

Current settings:
1. Show explored cells: ON
2. Show statistics: ON
3. Debug details: OFF
4. Show comparison maze results: ON
Enter setting number to change, or B to go back: 3
Debug details? [y/N], or B to go back: y
Setting updated.

Current settings:
1. Show explored cells: ON
2. Show statistics: ON
3. Debug details: ON
4. Show comparison maze results: ON
Enter setting number to change, or B to go back:
```

Figure WT2: Maze Solver Settings Menu and Changing Settings

You are also able to use the maze creator from the main menu through selecting option 2, which allows you to create your own maze row by row within the terminal, name the maze file and have it validated all in the same process.

```
Select an option: 2

Create a maze
Use # for walls, . for open cells, S for start, and E for goal.
New maze dimensions have a maximum of 100; larger mazes can be imported instead.
Enter maze height, or B to go back: 6
Enter maze width, or B to go back: 6
Enter row 1/6 (6 characters), or B to go back: #####
Enter row 2/6 (6 characters), or B to go back: #S####
Enter row 3/6 (6 characters), or B to go back: #.####
Enter row 4/6 (6 characters), or B to go back: #.####
Enter row 5/6 (6 characters), or B to go back: #...E#
Enter row 6/6 (6 characters), or B to go back: #####
Enter a name for the maze (without .txt), or B to go back: rightturn
Maze saved to mazes\rightturn.txt
It contains a solvable route of 6 moves.
```

Figure WT3: Maze Solver Maze Creator Process

From here, you can view the available list of mazes, as well as view each maze and their maze details. As you can see, our created rightturn.txt maze is immediately available for use, listed as number 18 on the list.

```
Select an option: 3

Available mazes
-----
1. blockedoff.txt
2. example.txt
3. malformed.txt
4. noend.txt
5. nostart.txt
6. nowall.txt
7. random1000_a.txt
8. random1000_b.txt
9. random100_a.txt
10. random100_b.txt
11. random100_c.txt
12. random20_a.txt
13. random20_b.txt
14. random20_c.txt
15. random50_a.txt
16. random50_b.txt
17. random50_c.txt
18. rightturn.txt
19. traditional1000.txt
20. traditional100_a.txt
21. traditional100_b.txt
22. traditional50_a.txt
23. traditional50_b.txt

Enter a maze number to view, or B to go back:
```

WT4: Maze Solver View Available Mazes List

```

Enter a maze number to view, or B to go back: 18
Preview: rightturn.txt

#####
#S####
# ####
# ####
#  E#
#####

Maze details: rightturn.txt
Dimensions: 6 x 6
Walls: 29
Walkable cells: 7
Start: (1, 1)   Goal: (4, 4)
Solvable: yes (6 moves)

Press Enter to return to the menu...|

```

WT5: Maze Solver Maze Preview and Statistics

Finally, the maze solver itself, firstly allows you to choose the desired maze to be solved through providing a list of the available mazes.

```

Select an option: 1

Available mazes:
1. blockedoff.txt
2. example.txt
3. malformed.txt
4. noend.txt
5. nostart.txt
6. nowall.txt
7. random1000_a.txt
8. random1000_b.txt
9. random100_a.txt
10. random100_b.txt
11. random100_c.txt
12. random20_a.txt
13. random20_b.txt
14. random20_c.txt
15. random50_a.txt
16. random50_b.txt
17. random50_c.txt
18. rightturn.txt
19. traditional1000.txt
20. traditional100_a.txt
21. traditional100_b.txt
22. traditional50_a.txt
23. traditional50_b.txt
Choose a maze (or B to go back):

```

WT6: Maze Solver List of Mazes

After choosing the desired maze, it will then show you a preview and ask if this is the maze you want to solve. This step was added so the user is able to see what the maze looks like instead of having to rely on memorising the name of their desired maze.

```
Choose a maze (or B to go back): 14
Selected maze: random20_c.txt

#####
# #           ##
# ### # ##### #
# # # # # # ##
### ### # ### ##
# # # # # # ##
# ### ### # ### ####
# # # # # # ##
# # ### ### # # # ##
# # # # # # ##
# ### # # ### # # ##
# #E# # # # # ##
# # # ### # ##### ##
# # # # # # ##
# ##### ### #####
# # # # S# # ##
##### ##### ##
# # # # # # ##
#####
#####
Solve this maze? [Y/n]:
```

WT7: Maze Solver Preview Confirmation

After confirming, the tool will ask the user which heuristic function to use. The user can choose a single heuristic, or choose all 3 and be provided with a comparison table of performance.

```
Solve this maze? [Y/n]: y

Choose solving method:
1. Manhattan distance (default)
2. Euclidean distance
3. Zero heuristic / Dijkstra
4. Compare all three
Select an option [1], or B to go back: |
```

WT8: Maze Solver Heuristic Function Choice

After choosing the desired heuristic function, the maze solver will then run and provide the user with the resulting maze and routes for each heuristic function, letting the user see which cells have been explored, which have not been explored, as well as what the final path chosen by the algorithm looks like.

```

Manhattan distance
-----
#####
# #,,,0000000000##
# ###,#o#####o##
#   #,ooo#,,,#o##
### ###,#o###,###o##
# #   #,#o#,,,#ooo##
# ### ##o#,,##o####
# #   #ooo#,,,#o#,,#
# # ###o###,##,#o#,,#
#   #,#o,,##,##,#o#,,#
# ###,#o#,,###,##,##
# #E#ooo#,,,,,#ooo##
# #o#o### #,####o##
# #ooo#   #ooo000##
# ##### ##o#####
#   #,,S#,,,,##
#####,#####,,#
#,,,,,,,,,,,,,,##
#####
#####

Euclidean distance
-----
#####
# #,,,0000000000##
# ###,#o#####o##
#   #,ooo#,,,#o##

```

WT9: Maze Results

Underneath the maze result visualisations, are the comparison statistics and debug information for the maze itself, if you have the setting for debug information turned on. This is helpful for locating the starting and ending points within larger mazes, as well as providing a general overview of the maze's components.

```
# # ###o###,#,#o#,#
# #,#o,,#,#,#o#,#
# ###,#o#,,###,#o#,#
# #E#ooo#,#,,,#ooo##
# #o#o###,#,#####o##
# #ooo#,,,#oooooooo##
# #####,###o#####
# # #,S#,,,,##
#####,######,##
#,,,,,,#,
#####
#####
#####

Heuristic comparison statistics
Method Path Explored Time (ms)
-----
Manhattan distance 56 123 0.290
Euclidean distance 56 123 0.236
Zero heuristic (Dijkstra) 56 128 0.205

=====
Debug information
=====
Dimensions: 20 x 20
Start: (15, 11)
Goal: (11, 3)
Walls: 239
Walkable cells: 161

Press Enter to return to the menu...|
```

WT10: Maze Solver Result Statistics and Debug Information

Reflection and Learning Experience

Something that surprised me was the fact that the A* algorithm wasn't always the fastest, despite being almost consistently the algorithm that explores the least amount of cells. In most cases, Dijkstra had the lowest time, whilst the results for Manhattan improved the bigger the maze was. My hypothesis for this is to do with the actual execution of the algorithm, as with Manhattan and Euclidean, the calculations for the heuristic functions take up time that Dijkstra's does not, as for Dijkstra, the $h(n)$ is always 0. Therefore, despite exploring less cells to find the solution and being more 'efficient', the runtime of Manhattan or Euclidean distance alongside the algorithm will be longer in some cases in comparison to Dijkstra's. This opens up considerations for whether or not the involvement of a heuristic function is suitable for the task at hand, or if the addition of a heuristic function is beneficial for the specific scenario you are looking to solve. And since these heuristics are better suited for different situations, the type of maze, edge weights as well as starting and ending locations may also have a significant effect on each heuristic function's performance. These are all great starting points for future research and development for this maze solver project, opening up the tool to more complex mazes with differing edge weights and special terrain.

Heuristic comparison statistics			
Method	Path	Explored	Time (ms)
Manhattan distance	40428	160267	331.763
Euclidean distance	40428	160273	331.583
Zero heuristic (Dijkstra)	40428	160303	330.249

Figure L1: Solved Maze random1000_a Results

Heuristic comparison statistics			
Method	Path	Explored	Time (ms)
Manhattan distance	40754	266087	566.095
Euclidean distance	40754	266109	584.635
Zero heuristic (Dijkstra)	40754	266211	543.862
Press Enter to return to the menu...			

Figure L2: Solved Maze random1000_b Results

Initial Mistakes & Errors

A major aspect of the project that I originally got wrong was the unchanging starting and ending points of the mazes. All of my generated mazes for trialing the maze solver tool had the starting position in the top left corner and the ending position as the bottom right corner. This thus hid the potential benefits of different heuristic functions as the general direction of search would always be towards the bottom right corner. However, this was one of the aspects I found out when the results always showed Dijkstra's having the best performance out of the three heuristic functions. Despite the static starting and ending positions not being the actual cause for the inconsistent performance across the heuristic functions, as I later realised, the functions themselves take time to process each cell as well as which frontier cells should be prioritised, the randomised starting and ending positions would then provide a more fair comparison between the different heuristic functions across different maze configurations. Another design aspect that I originally got incorrect would be the user interface, as I had not yet considered all the needs of the user that would be utilising the tool. The first iteration was clunky, having you go through each menu step by step, such as with the settings menu and not having any visualisations at all before solving the maze. Throughout the development process, I slowly started adding more and more features to the tool to make it more user friendly, such as previews of mazes before solving, debug information for the mazes to discern where the start and end was, better denotations for empty cells, explored cells and final chosen route, as well as different rendering for empty cells on the terminal to make the maze more readable for users.

What is something you understand now that you didn't before starting?

I now understand that simply exploring the least amount of cells is not the most time efficient and performant way of determining the best heuristic to be used. Instead, you would have to take into account the processing and calculation time of each heuristic. Despite Manhattan exploring the least amount of cells just about every single time, it usually had higher runtimes than Dijkstra's. Conversely, Dijkstra's always had the highest amount of cells explored, but the fastest runtimes. This is something that needs to be taken into account when thinking about which heuristic function to use in a real world scenario, what types of calculation would be most often performed, and future research into weighted edges and maze types should also be explored.

Another aspect that I had less understanding of compared to now would be the design for user interfaces as well as user experience overall. These changes most often came to mind as I was testing the maze solver tool, as I myself was the user of the tool in that instant. I would think to myself, 'Wouldn't it be nice if the settings menu didn't kick me back to the main menu after every single change I make?' And I would change exactly that. 'Wouldn't it be better if I had a preview of the maze I was selecting before it ran the solving process?' And I would then add the preview so next time I went to test the tool, I would know which maze I was picking without having to memorise the maze's name or to open up the maze folder and view it externally.

These new understandings have been very helpful when it comes to my mindset for considering software design, user interface and the user experience, as well as suitable uses for the algorithm, and the scenarios that would best fit each kind of algorithm or heuristic function.

AI Use & Experience

Which tools you used and how much

For this project, I used ChatGPT's Codex to help design most of the initial maze solver, A* algorithm implementation and test cases. I then iterated upon the base maze solver tool to improve the user mazes that experimentation was carried out on, user experience, user interfaces, overall workflow as well as debugging after every iteration.

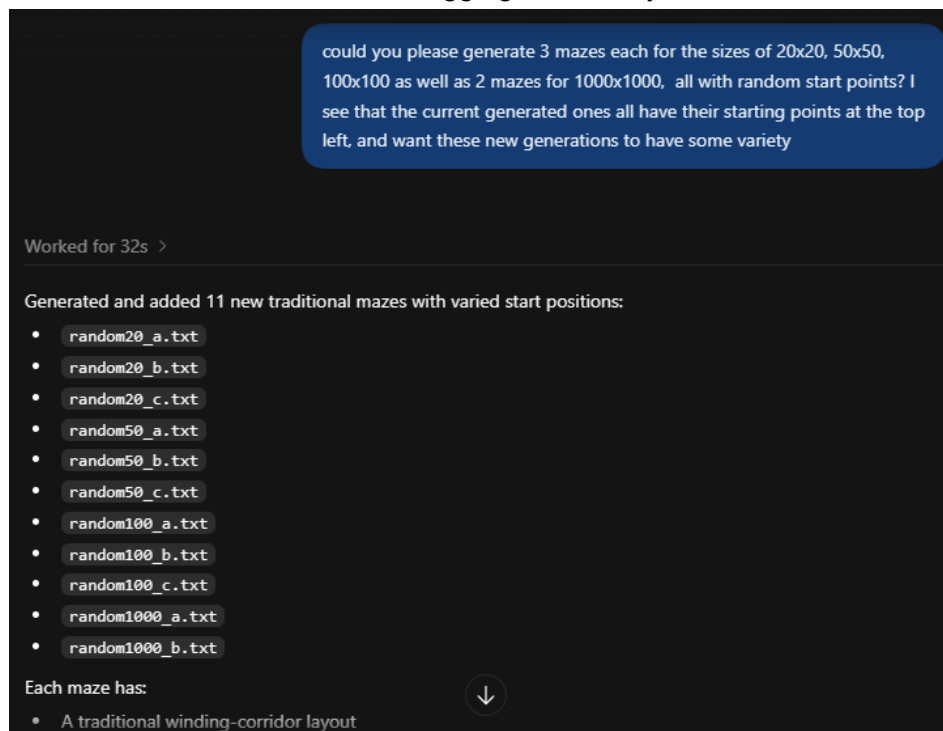


Figure AI1: Asking GPT to Generate Mazes to be Processed by the Tool.

Some examples of where the AI tool was wrong or unhelpful would be for the user interfaces, as often the required menu or maze information simply would not show up, and I would have to either ask ChatGPT what happened as the previous commit it was working, or I would have to implement the display of the information myself. One such example would be when the program would simply stop after a malformed maze was processed, GPT would tell me that it was intentional behaviour when running the tool directly from the terminal, but I was going through the CLI. This also taught me to be very specific in my inquiries to the AI, as it could misconstrue my questions and issues.

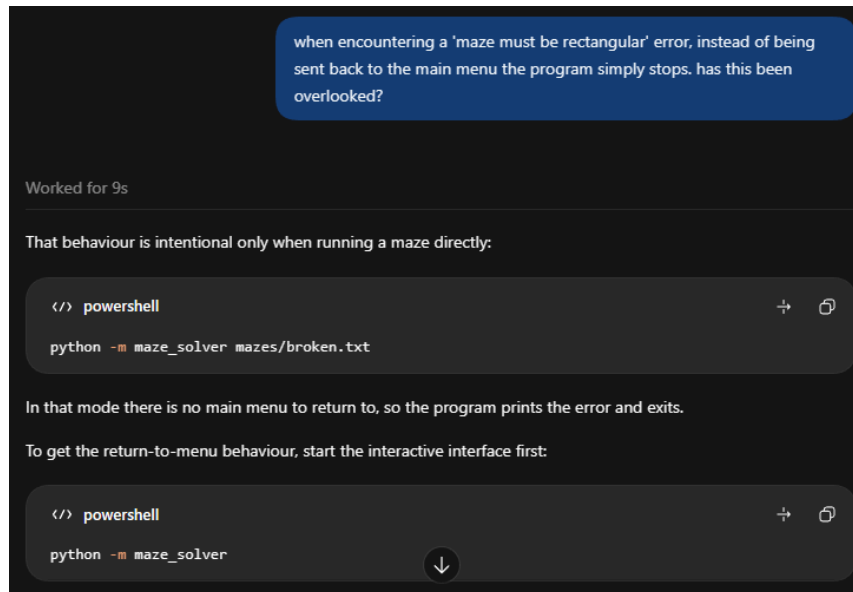


Figure AI2: GPT Misunderstanding the Issue due to Insufficient Information

Another example would be the implementation of the maze details for the view available mazes list. The details were only supposed to be displayed after you had selected a maze in order to keep the user interface tidy, but the AI tool placed the information directly on the selection menu, leading to an extremely cluttered and untidy interface. I had to then move the maze information code from the list portion of the program to the display maze portion in order to only have the maze information display after a maze was selected.

What I understood was how the A* algorithm and the heuristic functions worked, how the maze denotation and creation itself worked, as well as how the solver would display the maze and debug information. However, I took on trust in the AI tool's creation of the user maze creation feature and how it would take user input to build the maze directly in the terminal, and then to save it to the maze folder whilst providing input validation and file name validation to at the same time check if it was overriding an existing maze. This left me with more time to debug the maze solving as well as the displayed information.

References

A search algorithm* (2026) *Wikipedia*. Available at:
https://en.wikipedia.org/wiki/A*_search_algorithm (Accessed: 28 September 2026).

GeeksforGeeks (2026) *A* algorithm and its heuristic search strategy in AI*,
GeeksforGeeks. Available at:
<https://www.geeksforgeeks.org/artificial-intelligence/a-algorithm-and-its-heuristic-search-strategy-in-artificial-intelligence/> (Accessed: 28 September 2026).

Kumar, R. (no date) *The A* algorithm: A complete guide | datacamp*, *Datacamp*.
Available at: <https://www.datacamp.com/tutorial/a-star-algorithm> (Accessed: 28 September 2026).

Path finding - A with Manhattan distance or Euclidean distance for maze solving? - stack overflow* (no date) *Stack Overflow*. Available at:
<https://stackoverflow.com/questions/43906669/a-with-manhattan-distance-or-euclidean-distance-for-maze-solving> (Accessed: 30 September 2026).

Python - what is the purpose of the -M Switch? - stack overflow (no date) *Stack Overflow*. Available at:
<https://stackoverflow.com/questions/7610001/what-is-the-purpose-of-the-m-switch> (Accessed: 29 September 2026).

A Star Variants (no date) *Moving ai lab*. Available at:
<https://movingai.com/astar-var.html> (Accessed: 28 September 2026).